DataCheck

A browser-based CSV checker for the step before import

ASSESSMENT FORMAT	PROJECT TYPE
10-hour working plan Adapted from the supplied 3-day brief	Browser-based utility Static HTML, CSS and JavaScript
MAIN WORKFLOW	**REPORT DATE**
Choose file → check rows → review findings → export	4 November 2026

Executive summary

DataCheck is a browser-based tool for checking a CSV file before import. It flags common data-quality problems, lets the user inspect affected rows, and exports a clean subset alongside an issue list. The project is deliberately limited to one local-file workflow.

Project status

The application source and automated tests are included. The parser and validation tests pass (12 passed, 0 failed). A full browser walkthrough, cross-browser review and public deployment are still outstanding. This report keeps those limits visible rather than presenting them as completed work.

Prepared as a working project package. Review the code and run it locally before submitting it under your own name.

1. Opportunity & Problem

CSV files are often passed from one person or system to another before they are imported. A file can look fine when opened, but still contain blank cells, repeated identifiers, invalid dates or rows with a different number of columns. Those problems can lead to failed imports or extra manual checking later.

DataCheck is a first-pass check for that moment. The user selects one CSV file, reviews the rows that have been flagged and downloads two outputs: rows with no detected row-level issues, and a separate issue list. The file is processed in the browser; this version does not send it to an application server.

The need is a reasonable workflow hypothesis, not a proven customer finding. No user interviews or production observations were carried out for this project.

2. Exploration & Competitive Landscape

Three possible project directions were considered. The CSV checker was selected because it has a clear input and output, can be tested with sample data, and fits a short build window without requiring accounts or external services.

Direction	Why it was considered	Decision
CSV pre-import checker	A practical workflow with clear validation rules and testable results.	**Selected.** It offered the best balance of usefulness and achievable scope.
API response inspector	Could check status codes, response times and response structure.	Not selected. Network conditions and authentication would add variables to test.
Web asset audit tool	Could help developers inspect asset size and page loading.	Not selected. Reliable performance measurements would need a more controlled test setup.

A short review of official material for OpenRefine, CSVLint and Great Expectations helped set the boundary. Those tools offer broader data exploration, schema validation or repeatable pipeline checks. DataCheck is not intended to replace them; it focuses on a quick check of one file without setup. The research notes and links are in *03_EVIDENCE/research/competitive-landscape.md*.

3. Project Thesis & Scope

Project thesis: give someone handling a CSV file a quick way to spot common problems before importing it, without creating an account or sending the file to a server. The main workflow is intentionally small: select or drop a CSV file, run checks, filter the findings, and export the results.

In scope: local CSV selection, parsing, the listed validation checks, filtering, and two downloads. **Out of scope:** automatic data repair, custom schema authoring, accounts, saved history, team collaboration and production-system integration.

The checks cover blank cells, repeated complete rows, repeated values in a selected key column, email-like fields, date-like fields, inconsistent row widths, empty headers and repeated headers.

The parser handles quoted commas, escaped quotes, embedded line breaks, CRLF line endings and a UTF-8 byte-order mark. It rejects unterminated quoted fields and unexpected characters after a closing quote. The current file limit is 10 MB and the row limit is 25,000 data rows.

4. Engineering Strategy & Implementation

The project uses plain HTML, CSS and native JavaScript modules. There is no frontend framework or runtime JavaScript package. This keeps setup simple and makes the parser and validation rules easy to run in Node's built-in test runner.



Figure 1. The CSV file is read and processed in the browser. The local server only serves the application files.

The code is split into three main parts:

- **src/csv.js** reads CSV text into rows and serialises rows back to CSV.
- **src/validator.js** applies the data checks and returns findings for each row and for the file as a whole.
- **src/app.js** connects file selection, validation settings, filters, table rendering and downloads.

A few decisions are worth calling out. The user can choose the key column instead of the application assuming every file has an *id* field. Duplicate groups flag every occurrence, including the first row in the group. Date checks use a limited set of supported formats rather than guessing how an ambiguous date should be read.

5. Validation & Iteration

Automated test result: 12 passed, 0 failed. The tests cover CSV parsing, quoted fields, line endings, malformed input, CSV output escaping, missing values, duplicate rows and keys, invalid email and date values, inconsistent row widths, repeated headers and filtering.

The test command is `npm test`, run from *02_CODEBASE/*. JavaScript syntax checks for the three source modules also passed. A local static server returned HTTP 200 for the application page. The full test output is saved in *03_EVIDENCE/testing/npm-test-output.txt*.

Two changes came out of review. First, a separate notice was added for file-level problems such as empty or repeated column headers, because those findings do not belong to one data row. Second, the application was kept on system fonts so it does not need a remote font request.

The first test run also exposed a mismatch between a test expectation and the intended duplicate-key rule: all records in a repeated-key group should be flagged. The fixture was updated to include a clean row so the test checks both flagged and clean counts.

6. Limitations & Risks

The automated tests exercise the parser and validation logic. They do not prove that every button, filter or download works correctly in a real browser. Browser automation and screenshot capture were not available in the creation environment, so this package does not include fabricated UI screenshots or claim a full end-to-end test.

- The application has not been tested across multiple browsers.
- No accessibility audit or keyboard-only review has been completed.
- No large-file performance benchmark or security penetration test has been completed.
- No public deployment has been made; the supplied demo runs locally.
- Email checks inspect format only; they do not verify that an address exists.
- Date checks depend partly on column names and support a limited set of formats. Ambiguous US-style dates are not guessed.
- The application does not repair source values, save history, manage teams or connect to an import system.

These are the next things to verify, not features that should be described as finished.

7. Next Development Stage

- Run the app in a normal browser with several real but non-sensitive CSV files. Check upload, drag-and-drop, key selection, filters, reset and both downloads.
- Ask a few people who regularly handle CSV files to try the workflow. Use their feedback to decide which rules should be configurable.
- Add browser end-to-end tests for the main user actions and capture genuine screenshots.
- Test files of different sizes near the current limits and improve table rendering if it becomes slow.
- Complete keyboard, screen-reader and cross-browser checks before publishing a public version.

8. Running the project

Open a terminal in *02_CODEBASE/* and run:

```
python -m http.server 8000
```

Then open `http://localhost:8000` in a browser. Choose **Try sample data** or select a CSV file from the device. To run the tests, use `npm test` from the same folder. Full instructions are in *02_CODEBASE/README.md*.

9. Tool & Assistance Declaration

AI assistance was used substantially in planning the project, drafting and revising code and tests, reviewing test output, and preparing the report and supporting documents. The work should not be described as entirely unaided. Before submission, the person presenting it should run the project, inspect the code, understand the validation rules and be able to explain the decisions and limitations. The detailed declaration is included in *05_DECLARATION/AI_EXTERNAL_TOOLS.md*.

The application uses browser APIs and native JavaScript. Node's built-in test runner is used for tests, and Python's built-in HTTP server is used for the local demo. No third-party runtime JavaScript package is required.

References consulted

OpenRefine documentation: https://openrefine.org/docs/manual/facets and https://openrefine.org/docs/manual/transforming
CSVLint: https://csvlint.io/ and https://csvlint.io/about
Great Expectations: https://docs.greatexpectations.io/docs/core/define_expectations/create_an_expectation/ and https://docs.greatexpectations.io/docs/core/run_validations/